

PPML - RCWA

(Periodically Patterned Multi Layer – Rigorous Coupled Wave Analysis)

User manual



Simone Zanotto

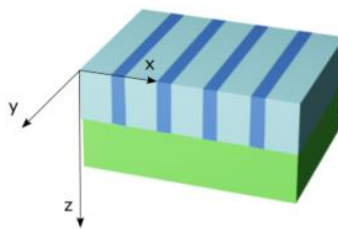
Version 4.0 – 12 june 2026

WARNING! PPML 4.0 is a major release, with many aspects revisited.

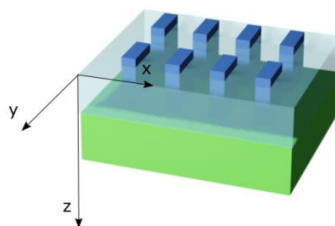
Please carefully check the current documentation.

PPML solves the scattering problem for layered systems with periodically patterned unit cells. It is thus suitable for modeling metasurfaces, metamaterials, photonic crystals, effective mediums...

The engine on which PPML is based is the Rigorous Coupled Wave Analysis, also known as Fourier Modal Method. Depending on the choice of geometry and materials, some differences in the solver are required. The first big choice is between **1d** and **2d** solvers. 1d solvers deal with structures that are invariant along one coordinate axis (in PPML it is the y axis):

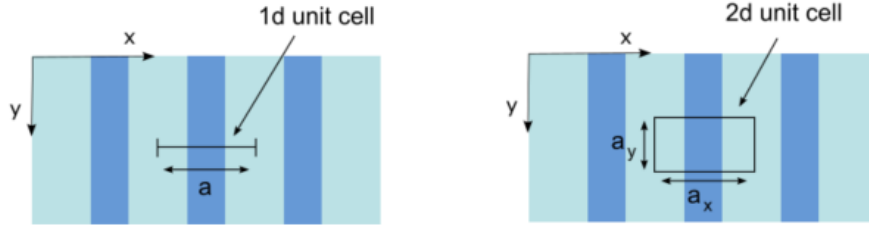


2d solvers deal with structures that have two-dimensional periodicity:



Unit cells can be rectangular or generally-shaped. Anisotropic materials can be handled, but Fourier factorization rules (FFRs) cannot be used. For 2d solver with FFRs, refer to PPML v3.0.

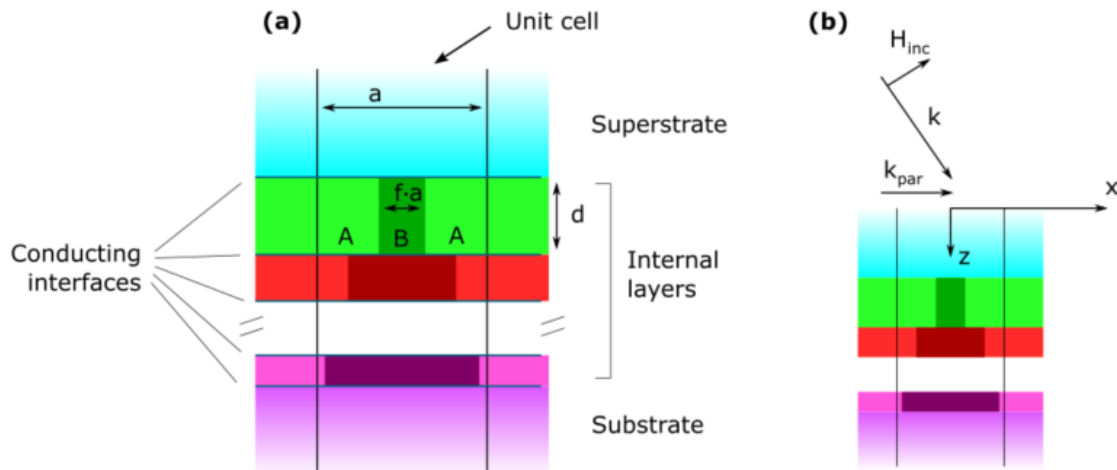
Note that a 1d structure can be simulated also with a 2d solver, just setting an appropriate pattern inside the unit cell:



The advantage to use the 1d solver is that it is faster; however, it is limited to analyze the case where the incident wave belongs to the xz -plane. Note that in this framework the polarizations are factorized, with TE (or s -pol) and TM (or p -pol) cases solved separately. See [1d PPML-TE](#) (folder **1d_te**) and [1d PPML-TM](#) (folder **1d_tm**) for details.

1d PPML-TE (folder 1d_te)

In the current implementation, the geometry is the following:



i.e. the unit cell consists of inclusions of material B inside a hosting material A. The inclusions are centered within the unit cell. The structure can have an arbitrary number of internal layers, and is capped by a superstrate and a substrate. The incident wavevector has modulus $k (=k_0 \cdot \sqrt{\epsilon_{sup}})$, being k_0 the vacuum wavevector and ϵ_{sup} the superstrate permittivity; moreover, the incident wavevector has a projection k_{par} along the x axis. The materials can have in principle anisotropic diagonal permittivity, however, the polarization is hard-coded to be “TE” (i.e. E field has only the y component), and hence the code reads a scalar value for the permittivity, which is the ϵ_{yy} entry of the aforementioned diagonal tensor. Complex-valued permittivities are allowed. The method works well also for strongly negative epsilons (metals), but care has to be taken in choosing the convergence parameter (**halfnpw**, see below). The layers are separated by zero-thickness, two-dimensional conducting interfaces, whose

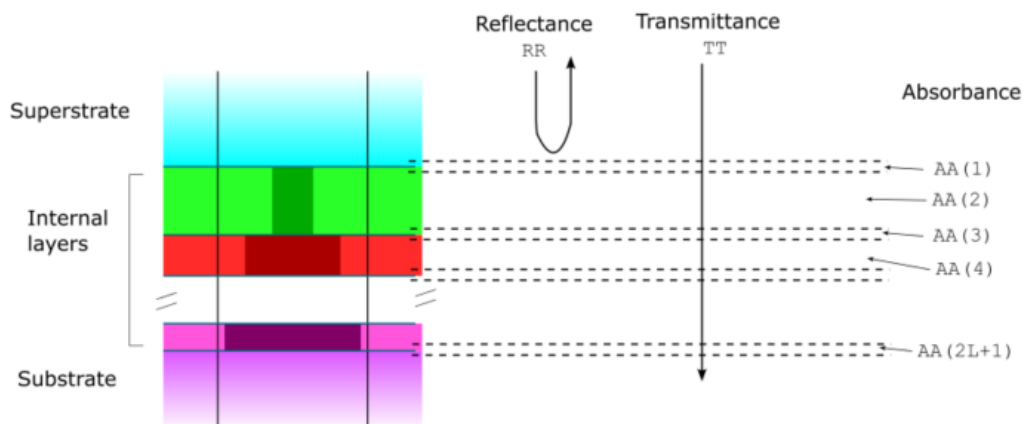
2d-conductivity can be specified as a complex number. Notice that setting the conductivity to zero is equivalent to assuming the direct contact between the dielectric materials of adjacent layers.

All the geometrical parameters are input as arguments of the function. See the comments in the function code and the examples.

The folder contains two functions: **RTA_1d_te**, **SM_1d_te**.

Function **RTA_1d_te**

Calculates the reflectance (reflected flux/incident flux), transmittance (transmitted flux/incident flux), and absorbance (absorbed power/incident flux). The absorbance is calculated layer-by-layer, and encoded in the vector **AA**. Each element of **AA** has contribution either from a conducting interface or from an internal layer. These are the reference planes for the calculations:



If ϵ_{pssub} is real, **TT** has the meaning of transmittance towards the far field.

If ϵ_{pssub} is complex, **TT** has the meaning of absorbance in the substrate.

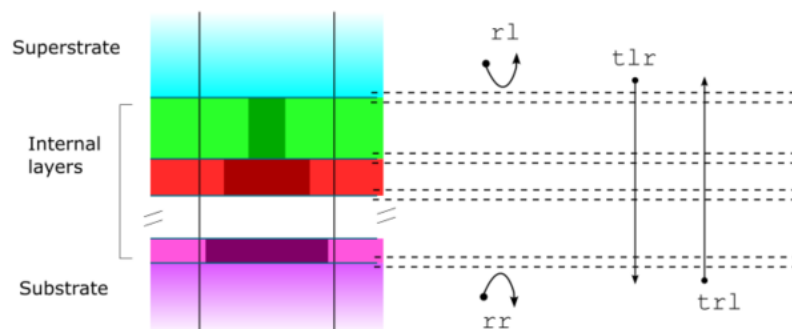
Above diffraction thresholds, **RR** and **TT** contain also the contribution of diffracted beams.

Thickness of superstrate and substrate do not influence the output of **RTA_1d_te**. In this calculation, super- and substrate are at all effects semi-infinite, independently of **d**.

Function **SM_1d_te**

The function calculates the amplitude reflection and transmission coefficients.

The coefficients connect the amplitudes of the E_y fields, calculated at the interface between superstrate and first layer, or substrate and last layer (outside of any conducting interface):



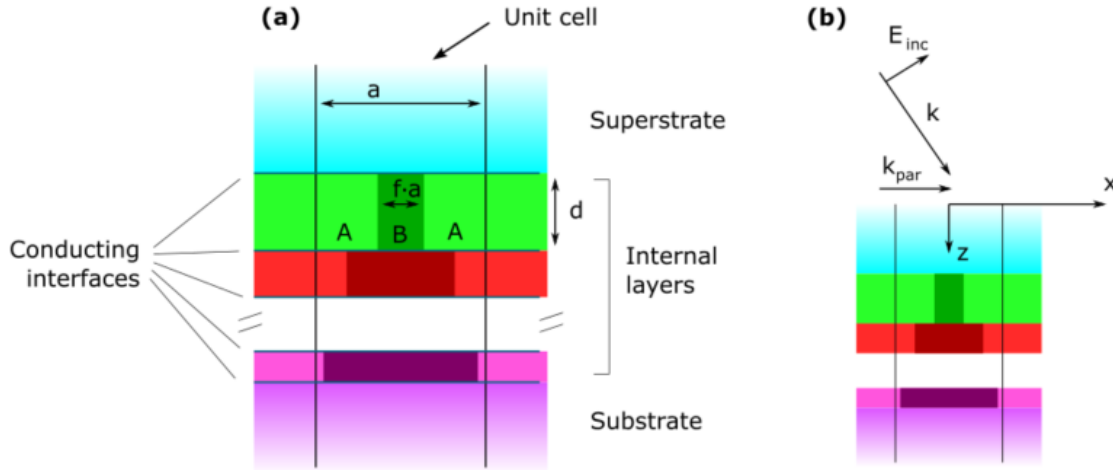
...Why “left” and “right” and not “up” and “down”? ...for historical reasons!

The setting of `halfnpw`

- As well known, the RCWA has a single convergence parameter which is the truncation order for the plane wave basis. Here, this is ruled by `halfnpw`, which is such that the complete basis is $(-\text{halfnpw}, \dots, 0, \dots, +\text{halfnpw})$.
Always pay attention that the selected value for `halfnpw` guarantees a satisfactory convergence for the problem under analysis.
- `halfnpw = 0` sets the number of harmonic waves to 1, e.g. the scattering matrix reduces to the ordinary 2x2 formalism for unpatterned multilayers

1d PPML –TM (folder 1d_tm)

In the current implementation, the geometry is the following:



i.e. the unit cell consists of inclusions of a material B inside a hosting material A. The inclusions are centered within the unit cell. The structure can have an arbitrary number of internal layers, and is capped by a superstrate and a substrate. The incident wavevector has modulus $k (=k_0 \cdot \sqrt{\epsilon_{sup}})$, being k_0 the vacuum wavevector and ϵ_{sup} the superstrate permittivity; moreover, the incident wavevector has a projection k_{par} along the x axis. Super- and substrate have isotropic permittivity, while internal layers can have anisotropic permittivity of the form

$$\begin{pmatrix} \epsilon_x & 0 & 0 \\ 0 & \epsilon_y & 0 \\ 0 & 0 & \epsilon_z \end{pmatrix}.$$

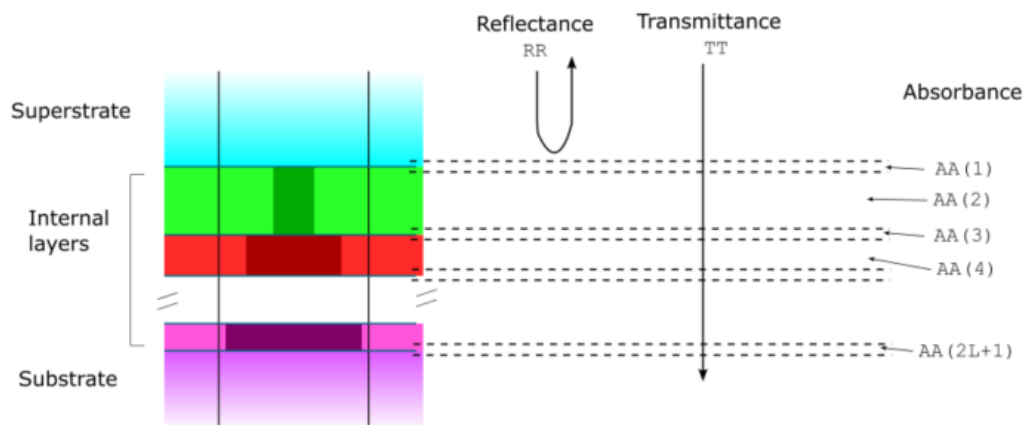
Due to the polarization factorization, only ϵ_x and ϵ_z are here relevant. Complex-valued permittivities are allowed. The method implements the Fourier factorization rules [Lifeng Li, J. Opt. Soc. Am. A **14**, 2758-2767 (1997)] and hence it works well also for strongly negative epsilons (metals), but care has to be taken in choosing the convergence parameter (**halfnpw**, see below). The layers are separated by zero-thickness, two-dimensional conducting interfaces, whose 2d-conductivity can be specified as a complex number. Notice that setting the conductivity to zero is equivalent to assuming the direct contact between the dielectric materials of adjacent layers.

All the geometrical parameters are input as arguments of the function. See the comments in the function code and the examples.

The folder contains three functions: **RTA_1d_tm**, **SM_1d_tm**, and **field_1d_tm**.

Function **RTA_1d_tm**

Calculates the reflectance (reflected flux/incident flux), transmittance (transmitted flux/incident flux), and absorbance (absorbed power/incident flux). The absorbance is calculated layer-by-layer, and encoded in the vector **AA**. Each element of **AA** has contribution either from a conducting interface or from an internal layer. These are the reference planes for the calculations:



If ϵ_{pssub} is real, **TT** has the meaning of transmittance towards the far field.

If ϵ_{pssub} is complex, **TT** has the meaning of absorbance in the substrate.

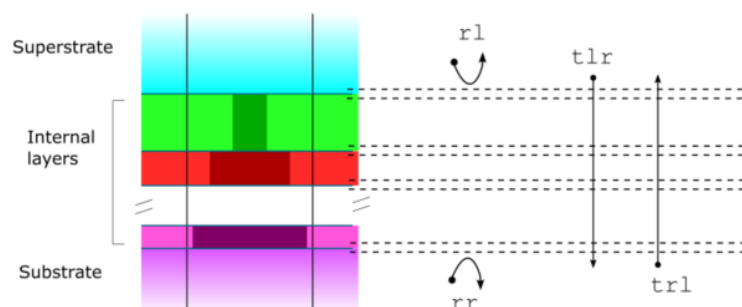
Above diffraction thresholds, **RR** and **TT** contain also the contribution of diffracted beams.

Thickness of superstrate and substrate do not influence the output of **RTA_1d_tm**. In this calculation, super- and substrate are at all effects semi-infinite, independently of **d**.

Function **SM_1d_tm**

The function calculates the amplitude reflection and transmission coefficients.

The coefficients connect the amplitudes of the H_y fields, calculated at the interface between superstrate and first layer, or substrate and last layer (outside of any conducting interface):



...Why "left" and "right" and not "up" and "down"? ...for historical reasons!



Function `field_1d_tm`

The function calculates the spatially resolved fields, over a grid calculated automatically by the function.

E fields are complex. The real, time-dependent fields are $\text{Re}(E \cdot \exp(-i \omega t))$ [$\omega = c \cdot k_0$].

The setting of `halfnpw`

- As well known, the RCWA has a single convergence parameter which is the truncation order for the plane wave basis. Here, this is ruled by `halfnpw`, which is such that the complete basis is $(-\text{halfnpw}, \dots, 0, \dots, +\text{halfnpw})$.
Always pay attention that the selected value for `halfnpw` guarantees a satisfactory convergence for the problem under analysis.
- `halfnpw = 0` sets the number of harmonic waves to 1, e.g. the scattering matrix reduces to the ordinary 2x2 formalism for unpatterned multilayers

2d PPML –basic general

Setting up the lattice

The **2d_basic_general** folder contains function aimed at studying structures with arbitrarily shaped unit cells with arbitrary permittivity distribution. The drawback of such generality is that the solver is based on ordinary (fast) Fourier transform, without the use of factorization rules or local coordinate transformations. Hence, it may be inadequate when materials with strong imaginary permittivity are investigated. Convergence tests are strongly recommended in any case.

The structure is composed of $L+2$ layers, where $l = 1$ is the superstrate and $l = L+2$ the substrate. Structures without internal layers can be simulated; in this case the structure consists just of a conducting layer between semi-infinite dielectrics. The z -axis zero is at the interface between the superstrate and the first internal layer (or the interface between superstrate and substrate, if no internal layers are declared). PPML adopts the convention such that the z -axis points from the superstrate to the substrate. Hence, looking into the structure from the superstrate, and keeping the x -axis pointing to the right, the y -axis will point downwards.

Superstrate and substrate must be homogeneous and isotropic. Each internal layer ($l = 2 \dots L+1$) can have its own specific permittivity distribution, however, all the layers must share the same unit cell basis vectors $\mathbf{a}_1$ and $\mathbf{a}_2$. The unit cell basis vectors do not need to be orthogonal, nor equal in modulus.

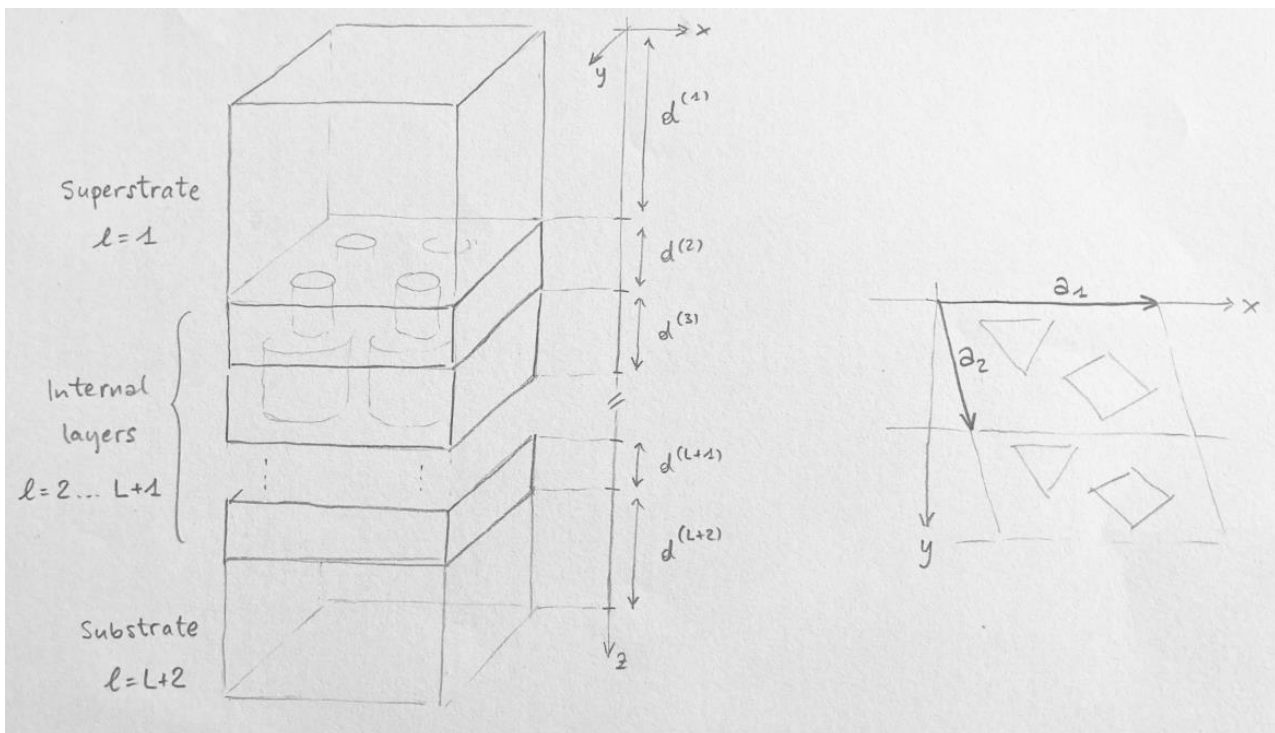


Figure 1. (left) 3d-schematic of a typical structure. (right) 2d-view of a possible choice for the unit cell.

Once defined the direct lattice unit cell, the reciprocal lattice unit cell is automatically defined, and it consists of basis vectors $\mathbf{g}_1^0$ and $\mathbf{g}_2^0$. Then, one must create a finite basis over the Fourier space to describe the electromagnetic fields. This is done by generating the vectors

$$\mathbf{g} = m\mathbf{g}_1^0 + n\mathbf{g}_2^0$$

where m and n are integer indices. The basis truncation scheme can be either parallelogramic

$$|m| < \text{trunc.rad1}, \quad |n| < \text{trunc.rad2}$$

or circular

$$|\mathbf{g}| < \text{sqrt}(|\mathbf{g}_1| |\mathbf{g}_2|) \cdot \text{trunc.rad}$$

resulting in the following schemes:

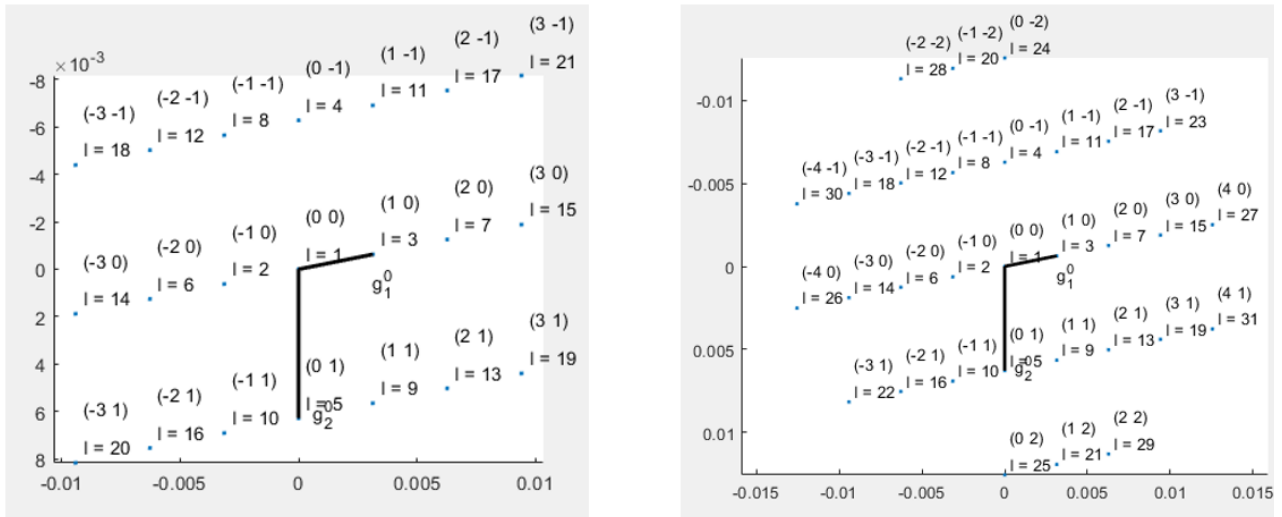


Figure 2. Reciprocal lattice for the parallelogram-like unit cell depicted in Fig. 1(right). The basis vectors are drawn in bold. The points of the reciprocal lattice are labeled by a single index I and by the double-index (n, m) (in parentheses). Left-side panel: parallelogramic truncation; right-side panel: circular truncation.

Here we exemplify the reciprocal lattice resulting from a direct space unit cell with the shape of a parallelogram, such as that sketched in Fig. 1.

To manage the basis, one has to introduce an ordering. PPML adopts a sorting based on the norm $|\mathbf{g}|$. This defines a new index I (a global index) as schematized in Fig. 2. Notice that the indexing is done ex-novo each time the `create_lattice` function is called, hence, the labeling of equal-modulus vectors can in principle be different at runtime.

The index I runs from 1 to N , which is the total number of plane waves (harmonics) that will be subsequently used for the RCWA calculation. The N value is available as `lattice.N`.

Moreover, `create_lattice` also generates an $N_1 \times N_2$ grid in the direct space unit cell. This should be used subsequently to create the Fourier transforms of the permittivity.

Description of the internal layer permittivity profile

The permittivity of internal layers can be anisotropic in the form

$$\begin{pmatrix} \varepsilon_{xx} & \varepsilon_{yx} & 0 \\ \varepsilon_{xy} & \varepsilon_{yy} & 0 \\ 0 & 0 & \varepsilon_{zz} \end{pmatrix}$$

where each tensor component has a (x,y) -dependence within the direct space unit cell. Let's focus on a specific internal layer. One hence has to define the functions $\varepsilon_{\alpha\beta}(x,y)$, and this is done by creating a structure, `epsmask`, having fields `epsmask.xx`, `epsmask.xy`, etc, each of which contains the permittivity tensor element evaluated over the direct space grid.

The structure `epsmask` can be created either by filling the variable by the user (such as in the examples `main_GeomPhase` or `main_Fan2017`) or using the function `create_epsmask_from_polygon` dedicated to binary structures. Once that `epsmask` is created, it can be plotted using `plot_epsmask`.

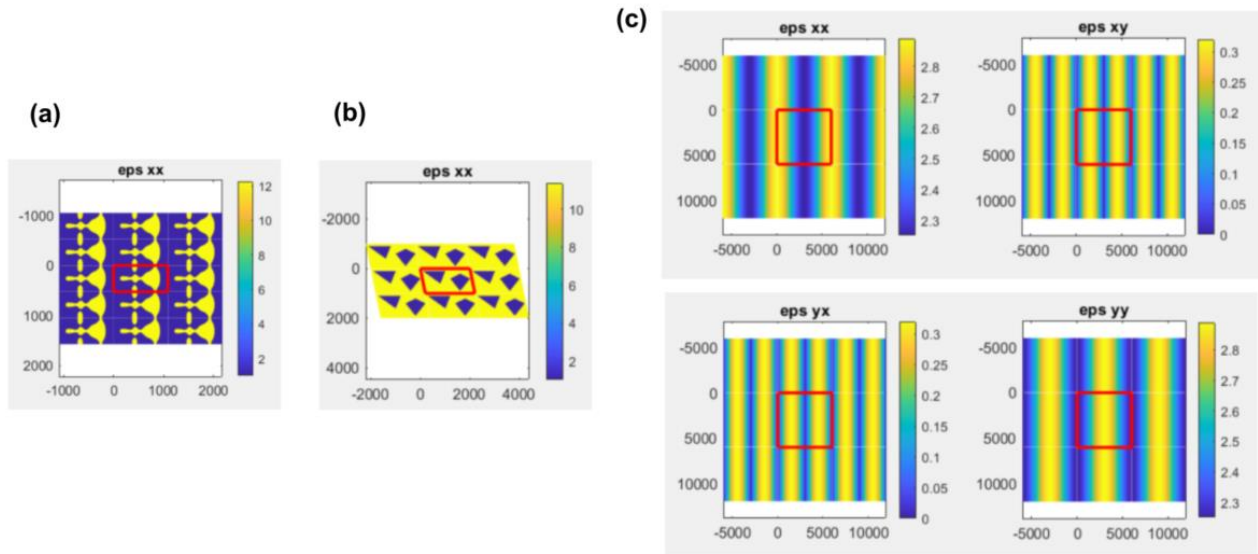


Figure 3. Permittivity distribution in the real space. Case (a) is a freeform, binary permittivity distribution, taken from the example `main_Fan2017`. Case (b) depicts polygonal inclusions in a parallelogram-shaped unit cell, from `main_ParallelogramUnitCell`. Case (c), taken from the example `main_GeomPhase`, shows the tensor component of the permittivity of a geometric phase liquid crystal grating.

The Fourier transform of `epsmask` is done by calling `create_fteps`, which uses the native MATLAB `fft2` function. It may happen that `create_fteps` raises an indexing error: in that case, consider increasing the real-space sampling parameters `N1` and `N2` in `create_lattice`, and the size of `epsmask.xx`, `epsmask.xy`, etc (whose size must match the aforementioned `N1` and `N2`).



Setting up the layer structure

The layer structure is described with `struc`, where the user must specify, layer by layer:

- the Fourier transforms of the layer permittivity (only for internal layer)
- the superstrate/substrate scalar permittivities
- the layer thicknesses (also for superstrate & substrate)
- the interface conductivities

PPML then automatically determines the number of layers. See also the comments in `solve_2d_basic_general` for further information.

Setting the global parameters & solving the RCWA algorithm

Being a frequency-domain method, PPML requires information on the operating frequency/wavelength/wavevector. Specifically, one must provide the free space wavevector as `globpars.k0`. There are no requirement on the physical units, but the units of `k0` must be the inverse units of the lengths that are provided for the direct cell base vectors, and for the layer thicknesses.

Following Bloch-Floquet theorem, PPML also requires the Bloch wavevector k_{par} , with x-y Cartesian components `globpars.kparx` and `globpars.kpary`. Usually this is provided considering that the structure is illuminated with a plane wave with incident wavevector k_{inc} , whose projection is indeed k_{par} . To define that, one should hence determine the incidence layer (super- or substrate), the incidence angles, and incorporate the proper permittivity in the definition. While we could have included this inside a PPML function, we preferred to keep the code lines in the main:

```
inlayer = 'sup'; % choose 'sup' or 'sub'
switch inlayer
    case 'sup'
        globpars.kparx = globpars.k0 * sqrt(struc.epssup) * sind(theta)*cosd(phi);
        globpars.kpary = globpars.k0 * sqrt(struc.epssup) * sind(theta)*sind(phi);
    case 'sub'
        globpars.kparx = globpars.k0 * sqrt(struc.epssub) * sind(theta)*cosd(phi);
        globpars.kpary = globpars.k0 * sqrt(struc.epssub) * sind(theta)*sind(phi);
end
```

In this way, we provide more flexible operation if one wishes, for instance, to perform sweeps in k_{par} not directly referring to the angles of incidence (such as in band structure calculation).

WARNING: If k_{par} is set to (0,0), the RCWA likely encounters singular matrices. Consider adding a small value (e.g., 1e-5), or analogously, use a finite incident angle (e.g., $\theta = 0.01$ degrees).

It is now time to run `solve_2d_basic_general`, which performs the core of RCWA calculations (i.e., solving the layer eigenvalue problems, and propagating the solution through the layer stack). This is the most time-intensive step. The output is the structure `wm` that contains the “working matrices”.

Calculating physical observables

In PPML v4.0 we provide three functions: **RTA_2d_basic_general**, **DJM_2d_basic_general**, and **field_2d_basic_general**.

Function **RTA_2d_basic_general**

This function calculates the output intensities in all the diffraction orders, and the energy absorbed in all the internal layers & conducting interfaces. The input wave is in the $I = 1$ Fourier wave, i.e. in the $(m,n) = (0,0)$ diffraction order. The user shall specify the input layer (i.e., super- or substrate), and the polarization.

The open diffraction orders can be identified graphically by setting `extra.plotflag = 1`, which produces an output like this in Fig. 4:

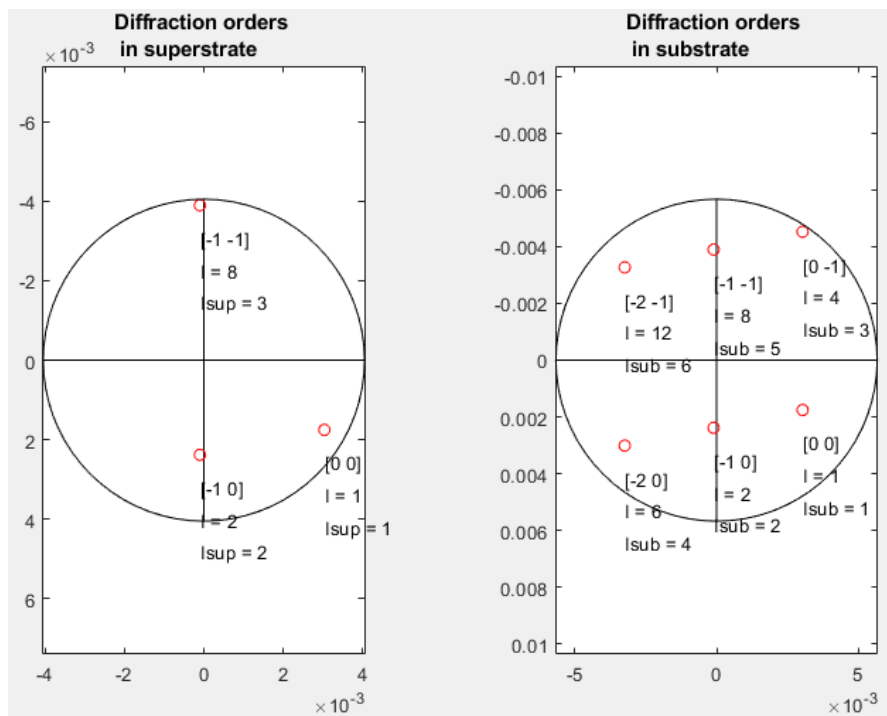


Figure 4: Map of diffracted orders (from the `main_ParallelogramUnitCell` example); see also Fig. 2.

Here, a new indexing is generated, and included within the `RTAdata` structure. In the superstrate, the `Isub` index runs from 1 to `Nsub` (which is the number of open diffraction channels in superstrate). The mapping between the `I` index and the `Isub` index is contained in `RTAdata.IIsub`, vector, whose `Isub`'th element is the corresponding `I` index. Notice that, if the $I = 1$ order is open, PPML sets `Isub = 1` for the $I = 1$ order. An analogous notation holds for the substrate.

The situation in Fig. 4 is a rather complicated one, due to the off-normal incidence set in that example (i.e., k_{par} was very different from zero) and to skewed unit cell. In many cases (such as in



`main_GeomPhase` or in `main_Fan2017`), the mapping between `Isup/Isub` and `I` is one-to-one, but please note that this is just a special situation.

The diffraction efficiencies are contained in `RTAdata.R` and `RTAdata.T`, where `R` and `T` refer to the reflection side and the transmission side (in turn, depending upon the choice of the incidence side, given in `incpars.inlayer`). `RTAdata.R` and `RTAdata.T` are indexed according to the global `I` indexing (that runs from 1 to `lattice.N`). The closed diffraction channel have an efficiency equal to zero, exception made for the case where the permittivity of the transmission side is complex (e.g., a metal substrate/superstrate – depending on the incidence side). In that case, the concept of diffraction threshold does not exist and all the elements of `RTAdata.T` are in general all populated with the intensity of the evanescent partial wave that is eventually dissipated in the lossy sub/superstrate.

When the open channels are defined it is convenient to list only their efficiencies. To do so, simply use the code

```
switch inlayer
  case 'sup'
    [RTAdata.R(RTAdata.IIsup), lattice.mn(RTAdata.IIsup,:)]
    [RTAdata.T(RTAdata.IIsup), lattice.mn(RTAdata.IIsup,:)]
  case 'sub'
    [RTAdata.R(RTAdata.IIsup), lattice.mn(RTAdata.IIsup,:)]
    [RTAdata.T(RTAdata.IIsup), lattice.mn(RTAdata.IIsup,:)]
end
```

This also retrieves the corresponding (m,n) indices. See the examples.

Function `DJM_2d_basic_general`

This function calculates the diffraction Jones matrix for each pair of input/output channel. First, the open diffraction channel in super- and substrate are determined, analogously to what happens in `RTA_2d_basic_general`. The indices are stored in the `difdata` structure. Then, the local polarization bases are determined, following the scheme in Fig. 5.

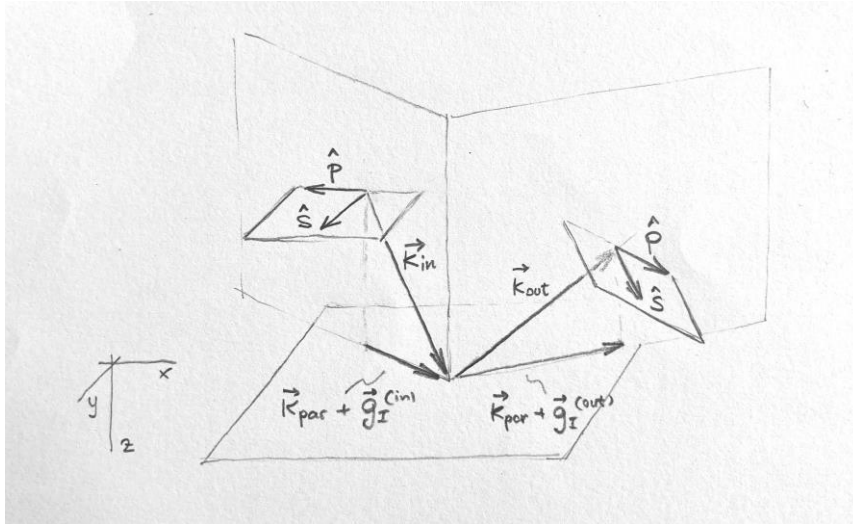


Figure 5: Local polarization bases for incoming and outgoing diffracted waves. $(\mathbf{s}, \mathbf{p}, \mathbf{k})$ always form a right-handed triplet. For incoming waves, $\mathbf{p}$ is set to be parallel to the corresponding $(\mathbf{k}_{\text{par}} + \mathbf{g}_I)$; for outgoing waves, it is antiparallel. The figure illustrates waves in the superstrate, but analogous definitions hold in the substrate.

Finally, the Jones matrices are calculated and stored in the `difdata` structure. The variable names contain information about the sides of wave incidence and output:

<i>Name</i>	<i>Indicence from</i>	<i>Output into</i>
supsup	superstrate	superstrate
supsub	superstrate	substrate
subsup	substrate	superstrate
subsub	substrate	substrate

Two different “types” of Jones matrices are calculated: $\mathcal{J}$ and $\mathcal{J}_t$. The $\mathcal{J}$ ’s contain the bare ratio of output to incident electric field, while the $\mathcal{J}_t$ ’s also keep track of the input/output plane wave angle, as well as of the medium wave impedance. In such a way, the $\mathcal{J}_t$ ’s are directly connected to energy flow and diffraction efficiencies.

Using the $\mathcal{J}_t$ ’s one can directly retrieve the results of `RTA_2d_basic_general`. For instance, these code chunks yield the same result (from example `main_GeomPhase`):

```
Ioutsub = 5; % output channel using the Isub indexing

incpars.Esp = [1;-1i];
RTAdata = RTA_2d_basic_general(wm,lattice,struc,globpars,incpars,extra);
efficiency_RTA = RTAdata.T( find( ~(RTAdata.IIsub - Ioutsub) ) )

difdata = DJM_2d_basic_general(wm,lattice,struc,globpars,extra);
efficiency_DJM = sum( abs( difdata.Jtsupsub{1,Ioutsub} * [1;-1i]/sqrt(2) ) .^2 )
```

Note that using DJM the Jones input vector must be normalized to one by the user.

The `find` operations are required since `RTAdata.T / RTAdata.R` use the full $I = 1 \dots \text{lattice.N}$ indexing, while DJM uses the reduced indexing for open diffraction channels ($I_{\text{sup}} = 1 \dots N_{\text{sup}}; I_{\text{sub}} = 1 \dots N_{\text{sub}}$), see Fig. 4.

The comparison could be done also relying on the (m,n) indexing; you can take inspiration from the example in the following section.

Function `field_2d_basic_general`

The function `field_2d_basic_general`, calculates the field inside the structure. The user shall specify which input channel (i.e. diffraction order) is excited, and with which polarization. There are three options about the domain slice where the field is calculated:

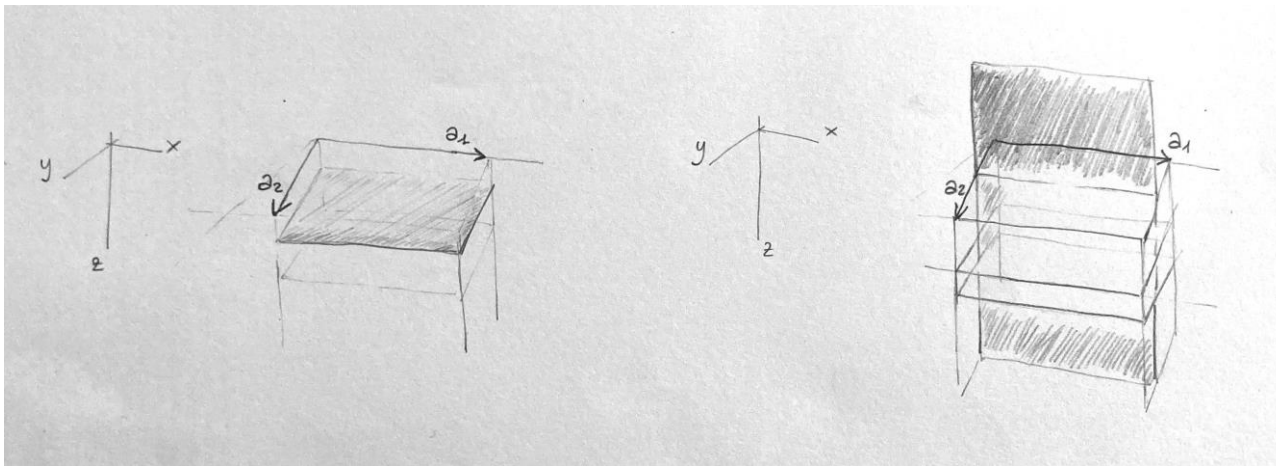


Figure 6: Types of domain sections available for field calculation. Left: a_1 - a_2 section. The user can specify the layer where to calculate the section, and the z -coordinate inside the layer. Right: a_1 - z section. The user can specify the coordinate along a_2 where to calculate the section.

The indexing of input channel follows the $I_{\text{sup}}/I_{\text{sub}}$ sorting described in Fig. 4. If the user prefers to specify the input channel with the (m,n) notation, the following code should be used:

```
mninc = [0,0]; % (m,n) indices of an open diffraction channel
switch inlayer
    case 'sup'
        incpars.Iinc = find( ~(difdata.IIsup - find(~vecnorm((lattice.mn-mninc),2,2))) );
    case 'sub'
        incpars.Iinc = find( ~(difdata.IIsub - find(~vecnorm((lattice.mn-mninc),2,2))) );
end
```

Here, `difdata` can be replaced with `RTAdata`, depending on which is available.

In PPML v4.0 only the E-field is calculated. Future versions will include also the H-field.

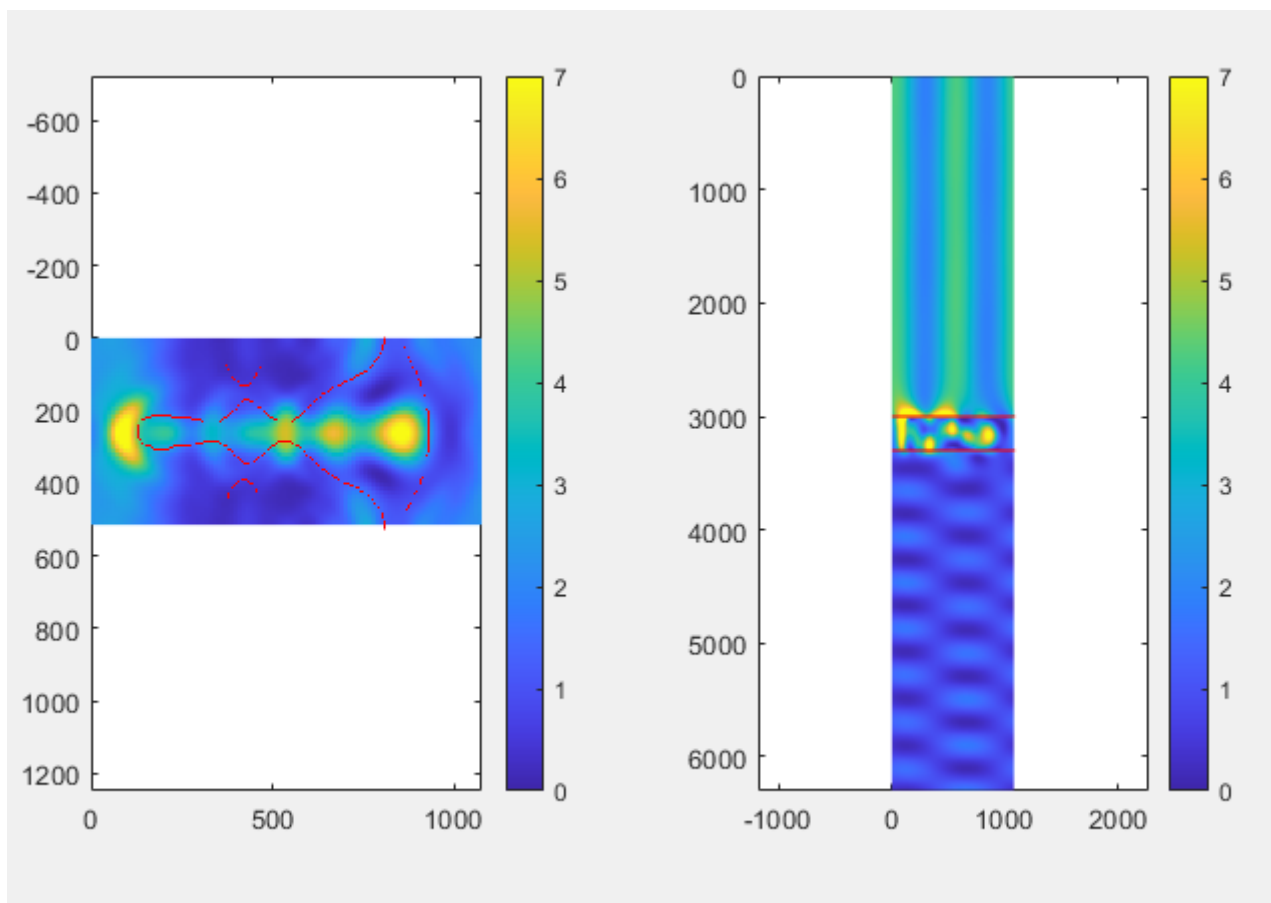


Figure 7: Example of field calculation (from main_Fan2017)